



So, What Is Requirements Work?

Neil Maiden

BELIEVE IT or not, I've been editing this column for more than eight years. I think that is long enough, and that it's time to give someone else a go. So, this is my last regular column for *IEEE Software*. I've tried to make it a really good one.

Back to Basics

I'm writing this column on a flight back from a requirements forum at which I took part in a panel. The theme of the panel was requirements engineering certification. Ever since I penned the *IEEE Software* column questioning the value of such certification,¹ I be-

lieve the essence of requirements work? If a requirements person could only do one thing for our projects, what would it be? I mean, if we're going to certify anything, surely we should certify the essence of requirements work.

During the panel and follow-up discussions, it became clear that forum participants wanted requirements people to be able to undertake various tasks such as eliciting, documenting, and managing requirements. This is all well and good, but is that what requirements work really comes down to? Scribing and cataloging?

Of course not.

Let's repeat that, so that we're all clear: requirements work involves locating and scoping problems, then creating and describing solutions for them.

Go on, say it out loud. Does it seem right? Even empowering?

Good.

So, this begs the question, why on earth do we make so much of our requirements work so diverse and complicated?

One answer concerns the complexity of what is often called *late requirements work*—work such as specifying, analyzing, and validating the specifications of large, evolving software-based systems. The scale and complexity of the software artifacts and their interactions with the world is what complicates this work.

In contrast, *early requirements work* should be more straightforward: this should locate and scope problems, and create some solutions. It's good old-fashioned problem solving—nothing more, nothing less.

Apollo 13 and Requirements

Let me remind you of a well-known incident. Most people know the story of Apollo 13. (Even if you aren't familiar with the original mission, you might have seen the Tom Hanks film portraying it.) After an explosion onboard the

If a requirements person could only do one thing for our projects, what would it be?

came the awkward squad-in-residence, invited to provide an opposing view to all of the certification bodies out there. This time, though, I tried not to be too provocative and avoided asking any inconvenient questions.

Rather, I thought, what is the real

In the end, requirements work is really about problem solving. Our primary function is to locate and scope problems, then create and describe solutions for them. The fact that much of this implementation work involves software is almost incidental.

Apollo 13 spacecraft, lithium hydroxide was needed to remove carbon dioxide from the spacecraft's atmosphere, but the design of the canisters available to filter the air were incompatible with the lunar module in which the crew were sheltering. To resolve this problem, engineers on the ground improvised a way to join cube-shaped canisters to cylindrical canister sockets using only spare equipment available in the spacecraft. In simple terms, these domain experts located the problem, created a solution, and communicated that solution to the crew via a radio so that it could be implemented.

Believe it or not, I consider this to be a great example of requirements work. A problem was located, then solved, and the solution specified so that it could be implemented.

Who Should Be Involved in Requirements Work?

So, going back to the requirements certification debate, can we really slap a certificate on such basic, intrinsic problem-solving skills? Can you ever imagine saying, "Sorry, I'm not going to listen to your otherwise eminently sensible solution to this problem because you haven't passed the prerequisite exam," or, "I know that you've been working in this business domain for 25 years, but because you don't know how to draw a use-case model, you're going to have to leave the room"?

Again, of course not.

Which leads me to another question: Who should actually be doing the problem solving at the heart of requirements work?

In many situations, an analyst who is parachuted into a project lacks the prerequisite domain knowledge to solve the problem him- or herself. Although he or she can bring a useful naivete to the problem, asking innocent questions such as why or what, it's rare that the analyst can locate and solve the prob-

lem alone. Rather, it's the people with the domain knowledge who are best positioned to solve it.

This means that, in many situations, the role of a requirements analyst is to facilitate the means for domain experts—who are often, but not always, the stakeholders—to locate and scope problems and create and describe solutions.

In essence, therefore, our role is to help others help themselves.

At this point, I am reminded of a keynote from an eminent Swedish academic who was talking some years ago about the number of computing graduates that Sweden generated each year, comparing this to the number of computing jobs advertised in that country during the same period. The number of tech-savvy people produced each year simply couldn't keep up with the demand. I accept that it's a bit of a leap, but I suspect this is the situation for requirements analysts worldwide—there simply aren't enough requirements analysts to locate, scope, and solve all of the problems out there.

So where does all of this leave the future of requirements work? Requirements analysts need to not only facilitate problem solving during early requirements work on individual projects but also enable stakeholders to solve more problems on their own over time. This means equipping and training domain experts with essential requirements competencies so that projects might need fewer and fewer specialist requirements analysts over time.

Inevitably, my position here challenges the requirements engineering certifiers. We need everyone, not just a certified subset, to be doing requirements work, and we need to recognize it as requirements work. These people would never consider the need to be certified in requirements engineering when working in their domain, their sector, their discipline.

Moreover, I believe that requirements work can offer knowledge and skills to other problem-solving areas. For example, creative problem-solving techniques are available for problem finding and goal analysis, but our requirements techniques offer even more, such as specifying complex solutions and enabling us to validate solutions before implementing them. So, let's be more confident—look outward, not inward—and encourage more people to do what we do during problem solving.

And that's that! The end of eight years of columns in this great magazine. What has both fascinated and surprised me over the years is how often I have looked to the column for information, insights, and humor. Looking back, several of the columns stand out, from the guest columns by guru Colin Codephirst to my characterization of software people as Spocks and Kirks. For me, it's been a blast, and I've loved it—even the constantly looming deadlines!

The very best of luck to my successor, Jane Huang—the readers and the column are in very capable hands. 📺

Reference

1. N.A.M. Maiden, 2012, "Oi Analyst—You're Barred," *IEEE Software*, vol. 26, no. 6, 2009, pp. 13–14.

NEIL MAIDEN is a professor of systems engineering and head of the Centre for HCI Design at City University London. Contact him at n.a.m.maiden@city.ac.uk.



See www.computer.org/software-multimedia for multimedia content related to this article.